Birla Institute of Technology & Science, Pilani
Hyderabad Campus

Computer Science & Information Systems

CSG527 Cloud computing – Project

Weightage: 15% ***Marks: 60*** ***Deadline: 19th April, 2026***

Design and Evaluation of an Intelligent, Scalable, and Cost-Aware Cloud Data Pipeline for Real-Time and Batch Analytics

Learning Outcomes

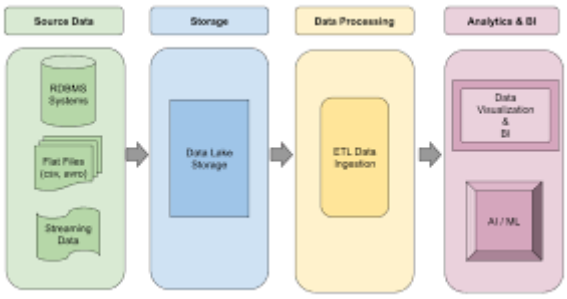
Students will:

- Design scalable cloud data pipelines
- Apply stream and batch processing paradigms
- Analyze cost–performance trade-offs
- Gain exposure to publishable cloud systems research
- Build industry-ready data engineering skills

1. Problem Background

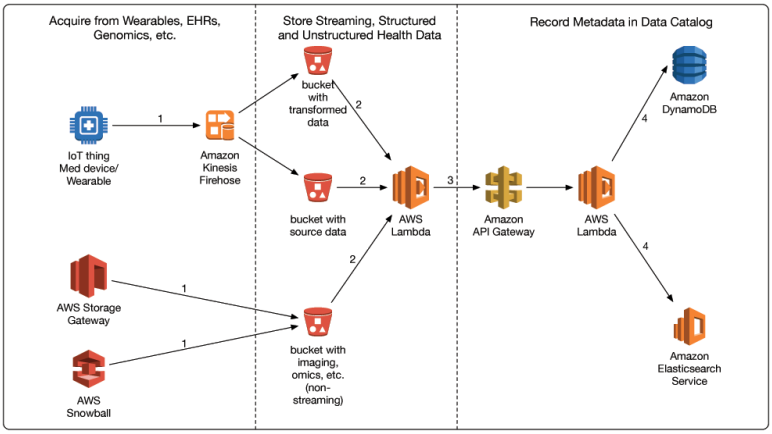
Enterprises today ingest data from heterogeneous sources such as IoT devices, application logs, transaction systems, and third-party APIs. These data streams exhibit high velocity, varying schema, data quality issues, and bursty workloads.

Traditional ETL pipelines fail to scale, incur high operational cost, and lack adaptability to changing workloads. Industry increasingly demands cloud-native data pipelines that support real-time + batch processing (Lambda/Kappa architectures) while ensuring low latency, reliability, observability, and cost efficiency.



From a research perspective, challenges remain in:

- Dynamic pipeline scaling
- Cost–latency trade-offs
- Adaptive data routing
- SLA-aware resource provisioning
- Intelligent fault handling



2. Problem Statement

Design, implement, and evaluate a cloud-native data pipeline platform capable of ingesting, processing, and storing both real-time and batch data while dynamically optimizing latency, throughput, reliability, and cost. The system should adapt to workload variability, data skew, and failure scenarios using intelligent orchestration and optimization strategies, and be deployable on a public cloud or open-source cloud stack.

3. Key Objectives

- Build a unified cloud data pipeline supporting real-time and batch analytics.
- Implement elastic scaling of ingestion and processing layers.
- Ensure exactly-once or effectively-once processing semantics.
- Incorporate cost-aware scheduling and resource allocation.
- Evaluate pipeline performance under realistic enterprise workloads.

4. Functional Requirements

4.1 Data Ingestion Layer

- Support multiple data sources:
  - Event streams (Kafka / PubSub / Kinesis)
  - Batch uploads (object storage)
  - REST / IoT feeds
- Handle schema evolution and late-arriving data.

4.2 Processing Layer

- Stream processing (Apache Flink / Spark Structured Streaming / Beam).
- Batch processing (Spark / BigQuery / Snowflake alternatives).
- Support windowing, joins, aggregations, and enrichment.

4.3 Orchestration & Deployment

- Use Kubernetes for pipeline orchestration.
- Infrastructure as Code (Terraform).
- Workflow scheduling (Airflow / Dagster).

4.4 Storage Layer

- Hot storage (NoSQL / in-memory).
- Cold storage (data lake on object storage).
- Analytical storage (columnar formats).

5. Non-Functional Requirements

Attribute	Requirement (Tentative)
Scalability	Auto-scale for 10× traffic spikes
Latency	Near-real-time analytics (<5s)
Reliability	≥ 99.9% pipeline uptime
Data Quality	Validation, deduplication
Cost Efficiency	Minimize cost per GB processed

6. Observability & Reliability

- End-to-end data lineage tracking.
- Pipeline metrics: throughput, lag, watermark delay.
- Failure detection and auto-recovery.
- Alerting and SLA violation detection.

7. Research & Innovation Opportunities

Students should explore at least one novel aspect, such as:

7.1 Cost–Latency Optimization

- Dynamic switching between spot and on-demand resources.
- Adaptive batch size and window tuning.
- Trade-off modeling between freshness and cost.

7.2 Intelligent Auto-Scaling

- Predictive scaling using workload forecasting.
- Reinforcement learning for resource allocation.
- Adaptive parallelism tuning.

7.3 SLA-Aware Data Processing

- Priority-based data routing.
- Admission control under overload.
- Graceful degradation strategies.

7.4 Fault Tolerance & Consistency

- Checkpoint optimization strategies.
- State recovery cost modeling.
- Failure-aware scheduling.

8. Evaluation Methodology (Academic Requirement)

Students must:

- Generate synthetic and real datasets.
- Conduct controlled experiments under workload variation.
- Compare baseline vs optimized pipeline strategies.
- Use metrics: latency, throughput, cost, recovery time.
- Perform statistical analysis of results.

9. Industry Relevance

This project aligns with real roles:

- Data Engineer
- Cloud Platform Engineer
- Big Data Architect
- MLOps / Analytics Engineer

Industry tools:

- Kafka, Flink, Spark, Airflow
- Kubernetes, Terraform
- Cloud-native data lakes
- Observability stacks

10. Deliverables

1. Architecture & Design Document
2. Source Code & Deployment Scripts
3. Performance & Cost Benchmarking Report
4. Research Analysis & Experimental Results
5. Final Demo & Presentation

11. Optional Future Extensions

- Multi-cloud data pipelines
- Privacy-aware data processing (PII masking)
- AI-assisted pipeline optimization
- Data pipeline self-healing mechanisms
- Green computing (energy-aware pipelines)

Guided Project Execution Steps (*Cloud Data Pipeline Capstone Project)*

Core Principle: *Focus on solving the problem first, then choose tools — not the other way around.*

Phase 0: Mental Model (Read This First – Very Important)

Before touching any technology, understand this simple pipeline story**:**

Data comes in → gets processed → gets stored → gets analyzed → system reacts

Every tool you use only does one of these things.

If you ever feel confused:

- Ask: *Which stage am I working on?*
- Ignore tools that don't help that stage.

Phase 1: Define the Problem (*No coding allowed in this phase)*

Step 1.1 – Pick a Simple Real-World Use Case

Choose **one** domain (do not mix domains):

Examples:

- E-commerce order events
- IoT sensor readings
- Online learning platform logs
- Bank transaction streams
- Healthcare device telemetry

Rule: If you can explain it to a non-technical person in 2 minutes, it's good.

Step 1.2 – Write the Problem in Plain English

Answer these in one-page max:

- What data is coming?
- How fast does it come? (slow / medium / bursty)
- What insights are needed?
- What happens if the system is slow or fails?

Rule: Avoid all tech terms here.

Step 1.3 – Define 3 Clear Success Metrics

Example:

- Data processing delay < 5 seconds
- Pipeline survives 5× traffic spike
- Cost per GB processed decreases under optimization

Phase 2: Draw the Pipeline (*Still no coding*)

Step 2.1 – Draw a Box Diagram

Use pen & paper or draw.io.

Data Source → Ingestion → Processing → Storage → Dashboard

That's it. No brand names yet.

Step 2.2 – Decide Batch vs Stream (Conceptually)

Answer:

- Does data need instant processing? → Stream
- Can it wait? → Batch
- Both? → Hybrid

Note: This decides architecture, not tools.

Phase 3: Choose Minimal Technology Stack (*Less is more*)

Step 3.1 – Choose One Tool Per Layer

Use this **safe default** if unsure:

Layer	Recommended
Ingestion:	Kafka / Cloud PubSub
Processing:	Spark or Flink
Storage:	Object Storage + Table
Orchestration:	Airflow
Deployment:	Docker + Kubernetes

Rule: Do NOT add tools unless a problem forces you to.

Step 3.2 – Lock the Stack

Once chosen:

- Do not switch tools mid-project
- Master only what you need

Phase 4: Build the Smallest Working Pipeline (*This is your safety net*)

Step 4.1 – Create Fake Data

Start with:

- Random data generator
- CSV or JSON
- 100–1000 records only

No real data yet.

Step 4.2 – Make Data Flow End-to-End

Goal: "Data enters → appears in storage"

Ignore:

- Scaling
- Optimization
- Cost

If this step works, **you will not fail**.

Phase 5: Add Cloud Deployment (*One cloud, one region*)

Step 5.1 – Containerize Everything

- One Docker image per component
- Use defaults

Step 5.2 – Deploy Once

- Single-node or small cluster
- No auto-scaling yet

Success = pipeline survives restart.

Phase 6: Observe Before Optimizing (*No improvement without measurement*)

Step 6.1 – Add Basic Monitoring

Track only:

- Input rate
- Processing delay
- Failure count

Graphs > logs.

Step 6.2 – Break the System (On Purpose)

- Stop a component
- Send too much data
- Kill a container

Write down:

- What broke?
- How long recovery took?

Phase 7: Introduce Intelligence (*This is where research begins*)

Choose **ONE** improvement:

- Auto-scaling
- Cost reduction
- Latency optimization
- Fault recovery
- SLA prioritization

Step 7.1 – Create a Baseline

Run pipeline **without optimization**. Record metrics.

Step 7.2 – Add Your Improvement

Examples:

- Predictive scaling
- Dynamic batch size
- Priority queues

Compare results.

Phase 8: Evaluation & Analysis (*Think like a researcher*)

Step 8.1 – Run Controlled Experiments

Change **one variable at a time**:

- Data rate
- Cluster size
- Failure frequency

Step 8.2 – Analyze Trade-offs

Answer:

- What improved?
- What got worse?
- Why?

Graphs matter more than code here.

Phase 9: Documentation & Presentation

Step 9.1 – Write Like a Story

Structure:

1. Problem
2. Architecture
3. Implementation
4. Experiments
5. Insights

Step 9.2 – Prepare the Demo

Demo flow:

1. Show data entering
2. Show metrics live
3. Trigger scaling or failure
4. Show recovery

Common Student Mistakes (As experienced in the past)

- Choosing too many tools
- Optimizing before measuring
- Copy-pasting cloud templates blindly
- Ignoring failure cases
- Overengineering early

Rule: Simple pipeline + strong evaluation = More Confidence and High Marks

Instructor Tip: A working simple pipeline with deep analysis scores higher than a complex broken system.